

Aufgabenblatt 04

Aufgabe 1

1. Aktueller Beitrag

Titel	Autor	Medium	Datum	Link
KI macht Entwickler ersetzbar, aber gute Architekten nicht	Von Golo Roden	https://www.heise.de/	03.12.2025	https://www.heise.de/blog/KI-macht-Entwickler-ersetzbar-aber-gute-Architekten-nicht-11097760.html

Das Thema ‚KI ersetzt Arbeitsplätze‘ ist extrem aktuell. Wir fanden diesen Artikel interessant, weil er keine Panik macht, sondern differenziert erklärt, dass sich unsere Aufgaben nur verschieben, statt komplett wegfallen.

2. Zusammenfassung

Der Artikel erklärt, dass LLM mittlerweile sehr gut Programmcode schreiben kann, wodurch reine Programmierer zunehmend ersetzbar werden. Allerdings fehlt der KI das Verständnis für komplexe Zusammenhänge und die Fähigkeit, sinnvolle Software-Strukturen zu planen. Deshalb werden Software-Architekten, die entscheiden, was und wie gebaut wird, in Zukunft wichtiger sein als diejenigen, die den Code nur abtippen. Kurz gesagt: Die KI übernimmt das Handwerk, aber der Mensch bleibt unverzichtbar für den Bauplan und die Strategie

3. Analyse des Artikels

Die neuen Möglichkeiten: Tempo und Demokratisierung

Technisch gesehen ist die Entwicklung beeindruckend. Tools wie GitHub Copilot oder ChatGPT nehmen Entwicklern lästige Fleißarbeit ab.

- Effizienz: Was früher Stunden dauerte (z. B. Standard-Funktionen schreiben oder Dokumentationen erstellen), passiert jetzt in Sekunden. Das macht Softwareentwicklung günstiger und schneller.
- Fokus auf das Wesentliche: Entwickler müssen sich weniger mit fehlenden Kommas oder Syntax-Fehlern herumschlagen. Sie haben mehr Zeit, um über die Logik und den Nutzen der Software nachzudenken.
- Zugänglichkeit: Auch Menschen mit weniger Erfahrung können nun einfache Programme erstellen, weil die KI ihnen bei der Übersetzung von „menschlicher Sprache“ in „Maschinensprache“ hilft.

Kritische Folgen und neue Probleme

Doch die Medaille hat eine Kehrseite, die im Artikel zwar angedeutet, aber oft unterschätzt wird. Wenn wir das Schreiben von Code komplett an die KI abgeben, entstehen neue Gefahren:

- Das Qualitäts-Problem (Halluzinationen):

KI versteht nicht wirklich, was sie tut. Sie rät statistisch, welches Wort oder Zeichen als Nächstes kommt. Das führt oft zu Code, der auf den ersten Blick perfekt aussieht, aber versteckte Sicherheitslücken oder logische Fehler enthält. Wenn der Mensch diesen Code nur noch „abnickt“, ohne ihn im Detail zu verstehen, bauen wir unsichere Software.

- Das Nachwuchs-Problem:

Das ist vielleicht der kritischste Punkt, den auch andere Experten oft anmerken: Ein „guter Architekt“ wird man nur, indem man jahrelang Code schreibt und dabei Fehler macht. Wenn die KI nun alle einfachen Aufgaben (die typischen Aufgaben für Anfänger) übernimmt, wie sollen Berufseinsteiger dann lernen? Es fehlt der „Übungsplatz“. Wir sägen also möglicherweise an dem Ast, auf dem die Experten der Zukunft sitzen sollen.

- Abhängigkeit und Urheberrecht:

Wenn Firmen sich blind auf KI-Tools verlassen (die meist großen US-Konzernen gehören), machen sie sich abhängig. Zudem ist oft unklar, woher die KI ihr Wissen hat. Hat sie Code gelernt, der eigentlich geschützt ist? Das führt zu rechtlichen Grauzonen.

Andere Quellen

Ähnliche Berichte, etwa von Stack Overflow oder Diskussionen auf Reddit, zeigen einen Trend: Die Anzahl der Fragen zu einfachen Programmier-Problemen sinkt, weil die KI sie beantwortet. Doch gleichzeitig warnen Sicherheitsfirmen, dass die Qualität von Code im Durchschnitt sinken könnte, weil immer mehr „Copy-Paste“-Code von KIs in Projekte fließt, ohne dass jemand die Risiken prüft. Der Tenor vieler Experten deckt sich mit dem Artikel: Die Rolle des Entwicklers verschwindet nicht, sie wandelt sich vom „Handwerker“ zum „Qualitätsmanager“.

Fazit und offene Fragen

Zusammenfassend lässt sich sagen: Der Artikel hat recht damit, dass reines Programmieren als Fähigkeit an Wert verliert. Aber die Schlussfolgerung, dass wir alle einfach „Architekten“ werden, ist zu einfach gedacht. KI ist wie ein extrem schneller Praktikant: Er arbeitet Tag und Nacht, erzählt aber manchmal Unsinn. Man braucht also mehr Fachwissen, um die KI zu kontrollieren, nicht weniger.

Folgende Fragen bleiben offen:

- Wer haftet, wenn eine KI fehlerhaften Code schreibt, der einen Schaden verursacht?
- Wie bilden wir in Zukunft junge Entwickler aus, wenn die einfachen Aufgaben zum Lernen wegfallen?
- Wird die KI irgendwann so gut sein, dass sie auch den „Architekten“ ersetzt, oder bleibt die kreative Gesamtplanung immer menschlich?

Aufgabe 2

1. Unterschied von RoPE und SPE

- Der Kernunterschied: Additiv vs. Multiplikativ

Der fundamentalste Unterschied liegt darin, wie die Positionsinformation in den Vektor eingebracht wird.

- **Sinusoidal Positional Encodings:** Sind additiv. Man berechnet einen Positionsvektor P und addiert ihn zum Wort-Embedding E .

$$X_{final} = E_{token} + P_{pos}.$$

Die Information "Ich bin an Position 5" wird auf den Inhalt "draufgelegt".

- **Rotary Positional Encodings:** Sind multiplikativ (rotierend). Man nimmt den Vektor (speziell die Query- und Key-Vektoren im Attention-Mechanismus) und rotiert ihn im Vektorraum um einen Winkel, der von der Position abhängt.

$$X_{final} = R_{pos} \cdot E_{token}.$$

Die Information "Ich bin an Position 5" ist nun in der Orientierung des Vektors kodiert.

- Mathematische Basis

- **SPE:** Hier basiert die Kodierung auf Sinus- und Kosinus-Wellen unterschiedlicher Frequenzen. Für eine Position pos und die Dimension i :

$$PE_{\{(pos, 2i)\}} = \sin(pos/10000^{2i/d})$$

$$PE_{\{(pos, 2i+1)\}} = \cos(pos/10000^{2i/d})$$

Diese festen Werte werden einfach addiert.

- **RoPE** nutzt die Eigenschaften komplexer Zahlen und die Eulersche Formel $e^{(i/\theta)} = \cos(\theta) + i\sin(\theta)$. Die Idee ist, den d -dimensionalen Vektor in $d/2$ Paare aufzuteilen. Jedes Paar (x_1, x_2) wird als komplexe Zahl $x_1 + ix_2$ betrachtet. Wenn ein Token an Position m steht, wird dieses Paar um den Winkel $m\theta$ rotiert. In Matrixform sieht die Rotation für ein Paar von Dimensionen so aus:

$$\begin{pmatrix} x'_1 \\ x'_2 \end{pmatrix} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Der gesamte Vektor wird durch eine Block-Diagonalmatrix rotiert, die aus vielen solcher 2x2-Rotationsmatrizen besteht.

- Vorteile von RoPE gegenüber SPE

Mathematische Trick der Rotation führt zu einer sehr mächtigen Eigenschaft im Attention-Mechanismus (dem Skalarprodukt zwischen Query und Key).

1. Natürliche relative Positionierung. Bei Transformern berechnet man die Aufmerksamkeit (Attention) durch das Skalarprodukt von Query (q) und Key (k). Wenn wir RoPE verwenden, passiert Folgendes mathematisch:

$$\langle R_m q, R_n k \rangle = q^T R_m^T R_n k = q^T R_{n-m} k$$

Das Ergebnis hängt nur noch von der Differenz (n-m) ab, also dem relativen Abstand zwischen den Token, nicht mehr von ihren absoluten Positionen (wie 5 oder 1005).

2. Bessere Extrapolationsfähigkeit (Length Generalization). Da die Kodierung auf relativen Abständen durch Rotation basiert, sind RoPE-Modelle oft stabiler, wenn sie auf Sequenzen angewendet werden, die länger sind als die im Training gesehenen (obwohl dies immer noch Tricks wie "NTK-Aware Scaling" oder "YaRN" erfordert, um wirklich gut zu funktionieren).

3. "Long-term Decay". Die mathematischen Eigenschaften von RoPE führen dazu, dass die Verbindung zwischen Token tendenziell abnimmt, je weiter sie voneinander entfernt sind (ähnlich wie unser Gedächtnis funktioniert). Dies ist bei rein additiven SPEs nicht so organisch gegeben.

- Nachteile von RoPE.

Trotz der Dominanz gibt es Nachteile:

Rechenaufwand: Eine Matrix-Vektor-Multiplikation (bzw. komplexe Rotation) ist rechenintensiver als eine simple Addition. In der Praxis ist dies jedoch vernachlässigbar, da spezialisierte Kernel (wie in FlashAttention) die Rotation extrem effizient durchführen.

Komplexität: Es ist schwieriger zu implementieren und zu debuggen als einfache Sinus-Additionen.

Dimension-Pairing: RoPE setzt voraus, dass Dimensionen paarweise behandelt werden. Das ist meist kein Problem, schränkt aber theoretisch die Freiheit der Vektorraum-Gestaltung minimal ein.

2. Siehe code

3. Beobachtung

```
1. Vektor-Norm (Länge) bei Position 5:
Original Norm: 8.6578
SPE (Additiv): 10.6115 (Verändert!)
RoPE (Rotativ): 8.6578 (Identisch/Erhalten)

-----

2. Shift-Test (Attention Score q @ k.T):
Wir verschieben das Paar (q, k) um 100 Positionen.

[Sinusoidal Additiv]
Score bei 0->5: 41.1248
Score bei 100->105: 46.4866
>> Abweichung: 5.3618 (Abhängig von absoluter Pos!)

[RoPE Rotativ]
Score bei 0->5: 10.7820
Score bei 100->105: 10.7820
>> Abweichung: 0.0000 (Perfekt stabil!)
```

1. Analyse der Vektor-Norm (Länge)

Ergebniss:

SPE: Norm steigt.

RoPE: Norm bleibt gleich.

Sinusoidal Encodings addieren Werte auf den Vektor. Das ist problematisch, weil es die "Bedeutung" des Vektors verwässern kann. Wenn der Positionsvektor zufällig sehr groß ist, überlagert er den Inhalt (das Wortembedding).

RoPE rotiert den Vektor nur. Die Länge des Pfeils im Vektorraum bleibt gleich, nur die Richtung ändert sich. Die "Information" (Länge) bleibt erhalten, sie wird nur kodiert. Das macht das Training stabiler.

2. Analyse der "Shift Invariance"

Ergebnis:

SPE: Score bei 0->5: 41.1248, Score bei 100->105: 46.4866. Unterschiedlich.

RoPE: Score bei 0->5: 10.7820, Score bei 100->105: 10.7820. Identisch.

Erklärung:

- Bei SPE (Additiv): Die Attention berechnet $(E_q + P_0) \cdot (E_k + P_5)$. Wenn wir verschieben, berechnen wir $(E_q + P_{100}) \cdot (E_k + P_{105})$. Da P_{100} ganz andere Werte hat als P_0 , kommen im Skalarprodukt "Mischterme" zustande (z.B. Inhalt mal Position), die von der absoluten Position (100) abhängen. Das Modell muss mühsam lernen, diese absoluten Werte zu ignorieren.
- Bei RoPE (Multiplikativ): Die Attention berechnet $(R_0 \cdot E_q) \cdot (R_5 \cdot E_k)$. Mathematisch vereinfacht sich das zu: $E_q \cdot R_{5-0} \cdot E_k$. Verschieben wir auf 100/105: $E_q \cdot R_{105-100} \cdot E_k$. Da $105-100 = 5$, ist die Rotation exakt dieselbe.

